

Techniques de compilation

Auditoire : CPI-2

Année Universitaire : 2025-2026

Code Classroom: **6xqrgwgl**

Dr.-Ing. Rakia Saidi

Maitre assistante en informatique
rakia.saidi@isimg.tn

Plan de ce cours

- **Chapitre 1**: Introduction à la compilation
- **Chapitre 2**: Rappels à la théorie des langages
- **Chapitre 3**: Analyse lexicale
- **Chapitre 4**: Analyse syntaxique
- **Chapitre 5**: Analyse sémantique
- **Chapitre 6**: Génération de code

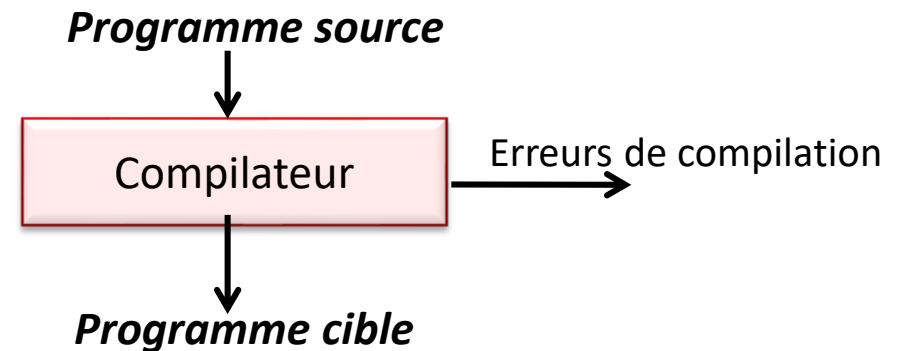
Chapitre I : Introduction à la compilation

Plan

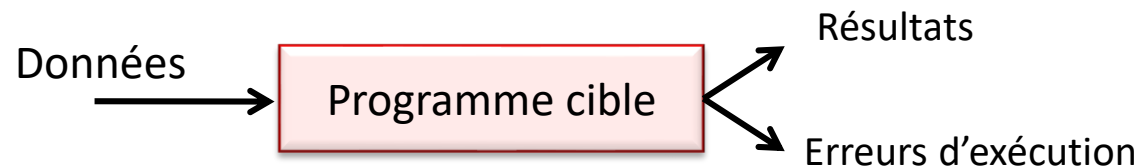
1. Qu'est ce que le compilateur ?
2. Les types de langages de programmation
3. Les étapes de compilation
4. Outils de construction de compilateur
5. La structure d'un compilateur

1. Qu'est ce que le compilateur?

- Un **compilateur** est un logiciel qui lit un programme, rédigé dans un **langage** source, et qui le **traduit** en un programme équivalent rédigé dans un autre langage (le langage cible)



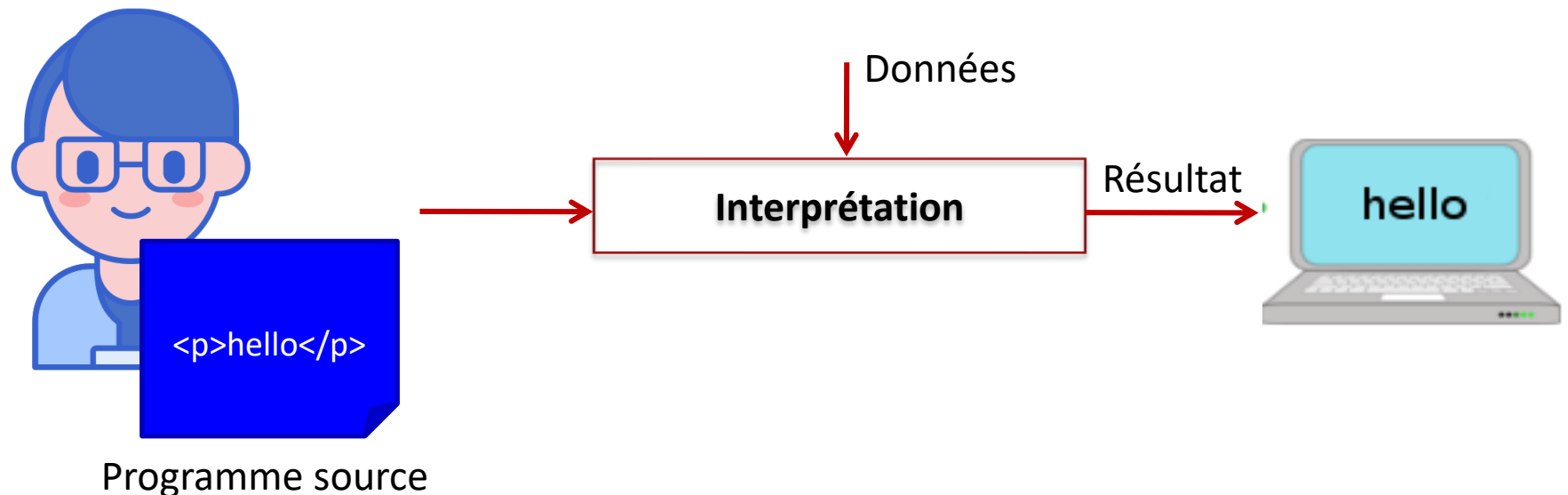
- Le compilateur doit signaler toute **erreur** (Construction syntaxique incorrecte, Identificateurs non déclarés,...) qu'il détecterait dans le programme source
- Si le programme cible est un programme exécutable en langage machine, l'utilisateur peut ensuite le lancer afin de traiter des données et de produire des résultats



2. Les types de langages de programmation

- **Langage interprété :**

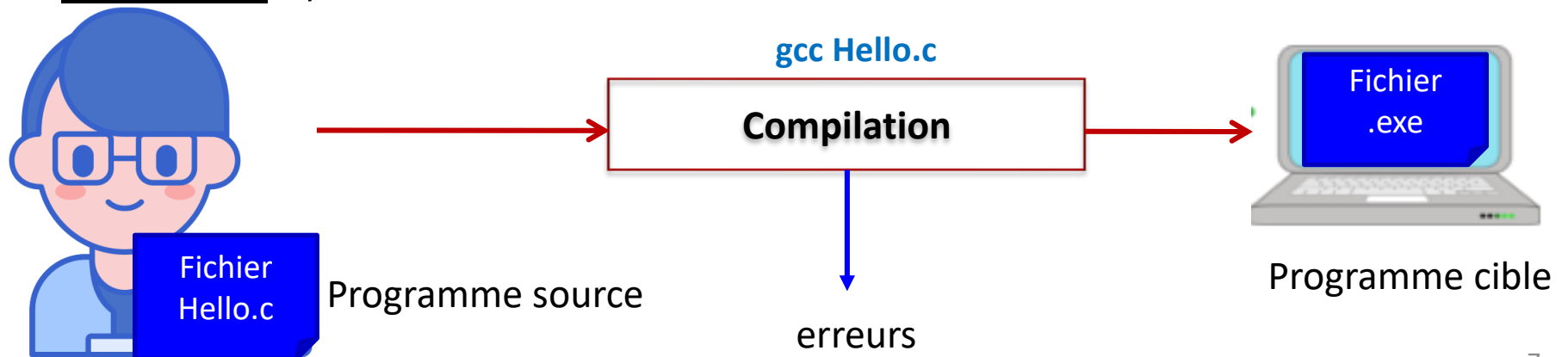
- Un programme écrit dans un langage **interprété** a besoin d'un programme auxiliaire (l'interpréteur) pour traduire au fur et à mesure une instruction (ou un bloc d'instructions) du programme.
- Il analyse une instruction après l'autre puis l'exécute immédiatement.
- Exemples: HTML, PHP, JavaScript



2. Les types de langages de programmation

- **Langage compilé :**

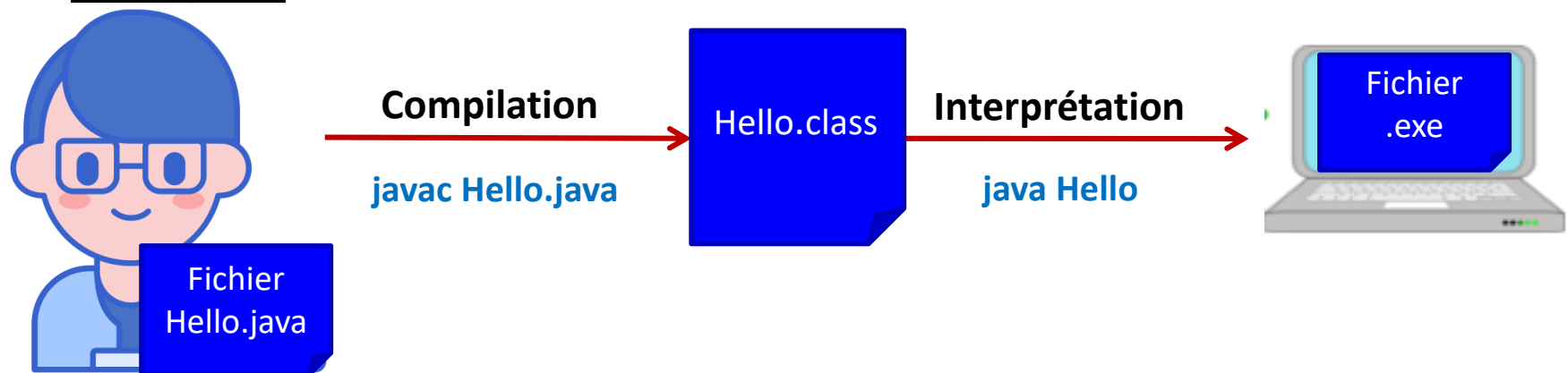
- Un programme écrit dans un langage **compilé** est entièrement transformé en un programme écrit en langage compréhensible par la machine
- Le fichier généré est autonome. C'est-à-dire qui n'aura plus besoin d'un programme autre que lui pour s'exécuter ; on dit d'ailleurs que ce fichier est **exécutable**.
- Exemples: C, Pascal



2. Les types de langages de programmation

- **Langage compilé et interprété (intermédiaire):**

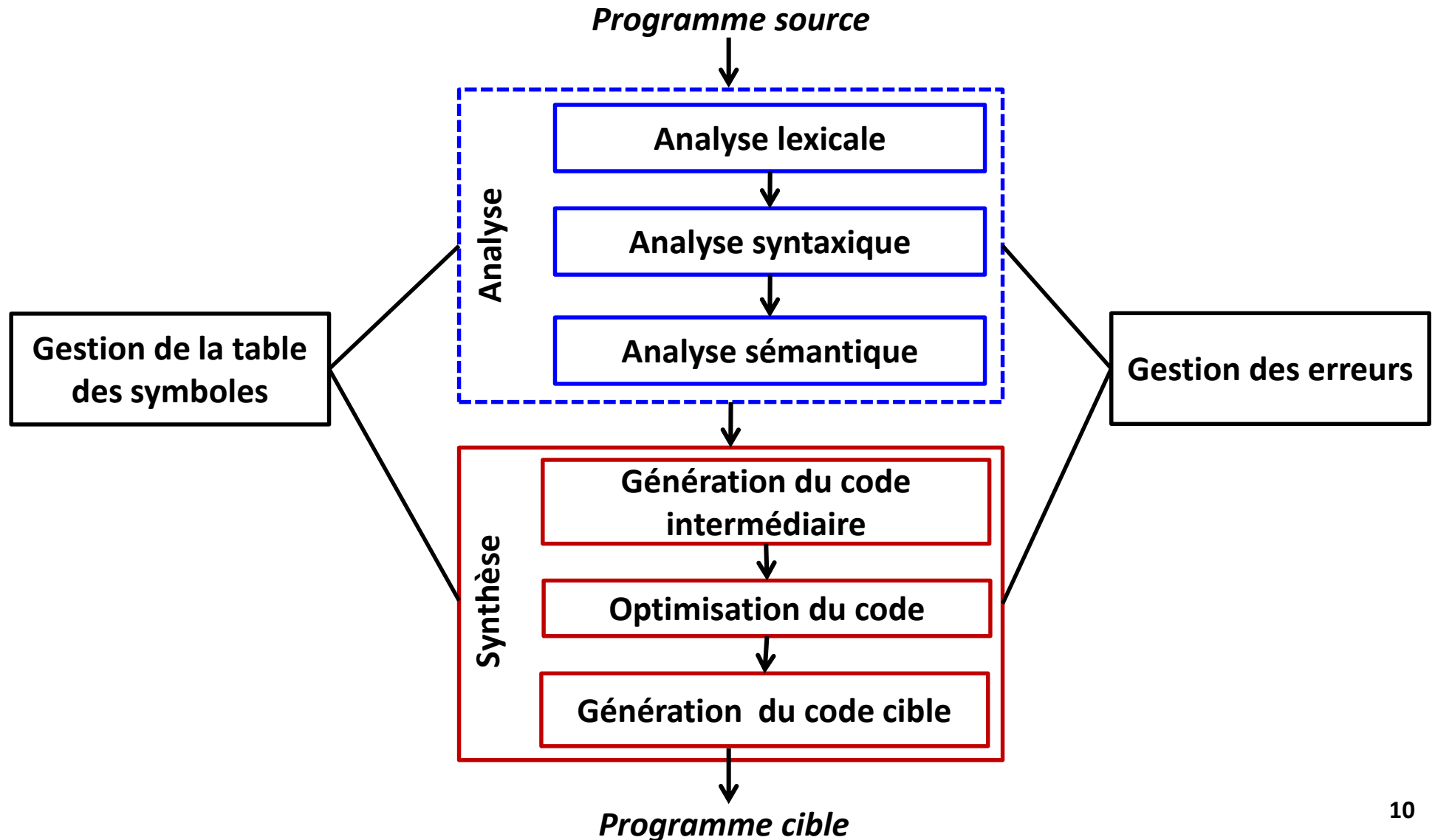
- Un programme compilé et interprété sera traduit (compilé) dans une forme binaire compacte (du pseudo-code ou P-code) qui n'est pas encore du code machine.
- Les langages intermédiaires (P-code) sont des langages à mi-chemin de l'interprétation et de la compilation.
- À l'exécution du programme, le P-code sera interprété.
- Exemples: Java et C++



3. Les étapes de la compilation

- La compilation se décompose en deux phases:
 - L'analyse permet de:
 - décomposer le programme source en ses constituants, tout en associant à chacun une structure grammaticale.
 - reconnaître les variables, les instructions, les opérateurs et élabore la structure syntaxique du programme ainsi que certaines propriétés sémantiques.
 - créer une représentation intermédiaire du programme source
 - accumuler diverses informations dans la table de symboles
 - La synthèse permet de:
 - construire le programme cible désiré à partir de la représentation intermédiaire et des informations de la table de symboles

3. Les étapes de la compilation



3. Les étapes de la compilation

Table des symboles

- C'est une structure de données contenant un enregistrement pour chaque identificateur. Il sert à stocker les informations qui concernent les identificateurs du programme source.
- Pour les attributs, la table de symboles garde:
 - Son emplacement mémoire, son type, sa portée...
- Pour les fonctions, la table de symboles garde:
 - le nombre et les types de ses arguments
 - le mode de passage de chacun d'eux et la valeur de retour
- Le remplissage de cette table aura lieu lors de la phase d'analyse.
- Ces informations seront utilisées lors des analyses syntaxique et sémantique, ainsi que lors de la génération de code.

3. Les étapes de la compilation

Gestion des erreurs

- Chaque phase peut rencontrer des erreurs. Il s'agit de:
 - Détecter les erreurs et informer le programmeur avec le maximum de précision (erreur de syntaxe, de sémantique ou erreur système)
 - Traiter les erreurs de telle manière que la compilation puisse continuer et que d'autres erreurs puissent être aussi détectées
- Un compilateur qui s'arrête à la première erreur n'est pas trop performant
- Un très grand nombre d'erreurs peuvent entraîner l'arrêt de l'exécution du compilateur

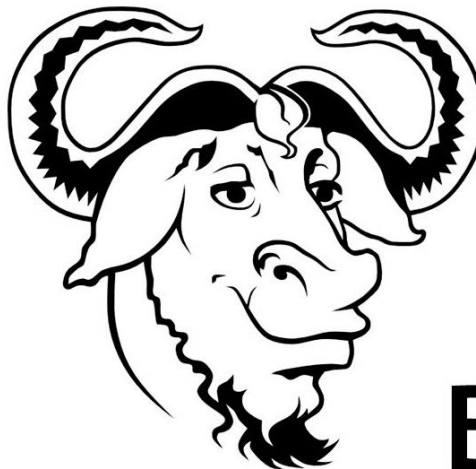
4. Outils de construction de compilateur

- Des outils spécialisés ont été développés pour faciliter l'implémentation de certaines phases des compilateurs
 - *Le constructeur d'analyseurs lexicaux* produit automatiquement un analyseur lexical à partir de la description des unités lexicales du langage par des expressions régulières
 - *Le constructeur d'analyseurs syntaxiques* produit automatiquement un analyseur syntaxique à partir d'une description d'un langage de programmation sous forme d'une grammaire non contextuelle
 - *Le moteur de traduction dirigée par la syntaxe* produit un ensemble de procédures pour parcourir l'arbre d'analyse en produisant du code intermédiaire
 - *Le constructeur de générateurs de code* produit un générateur de code à partir d'un ensemble de règles décrivant pour chaque opération du langage intermédiaire, comment la traduire dans le langage de la machine cible

4. Outils de construction de compilateurs

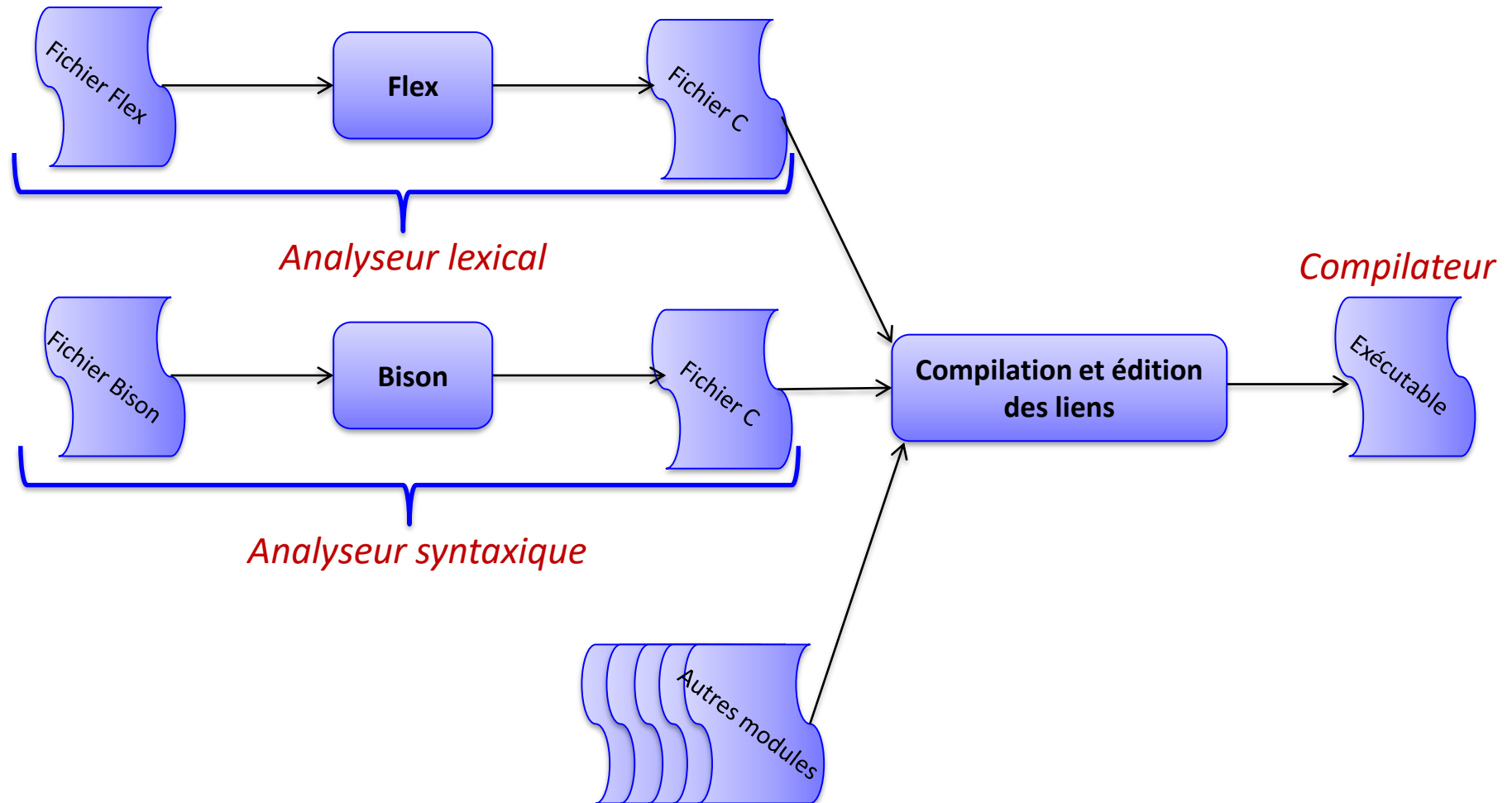
- Ces outils utilisent des langages spécialisés pour la spécification et l'implémentation de composants de compilateur spécifiques
 - Exemples:
 - Un constructeur d'analyseurs lexicaux appelé *Flex*
 - Un constructeur d'analyseurs syntaxiques appelé *Yacc*

Flex

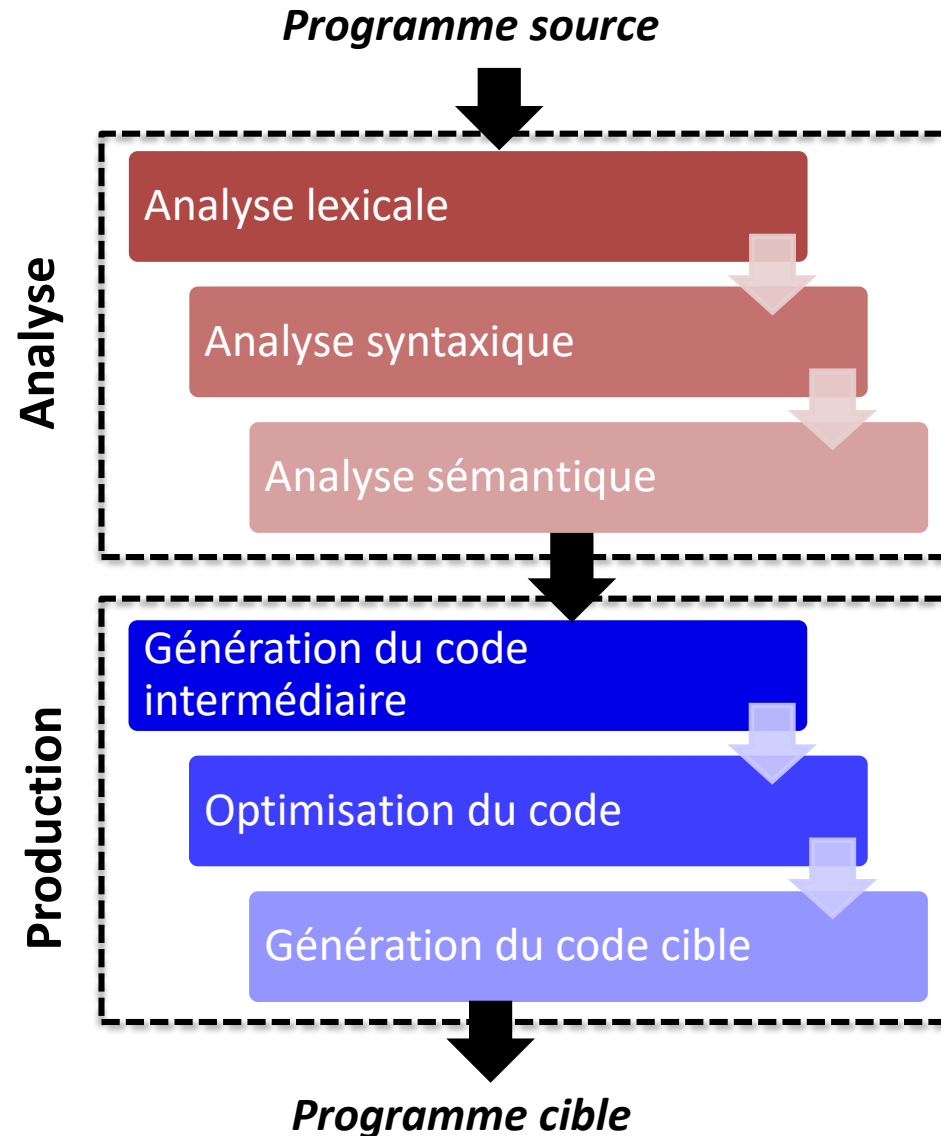


Bison

4. Outils de construction de compilateurs



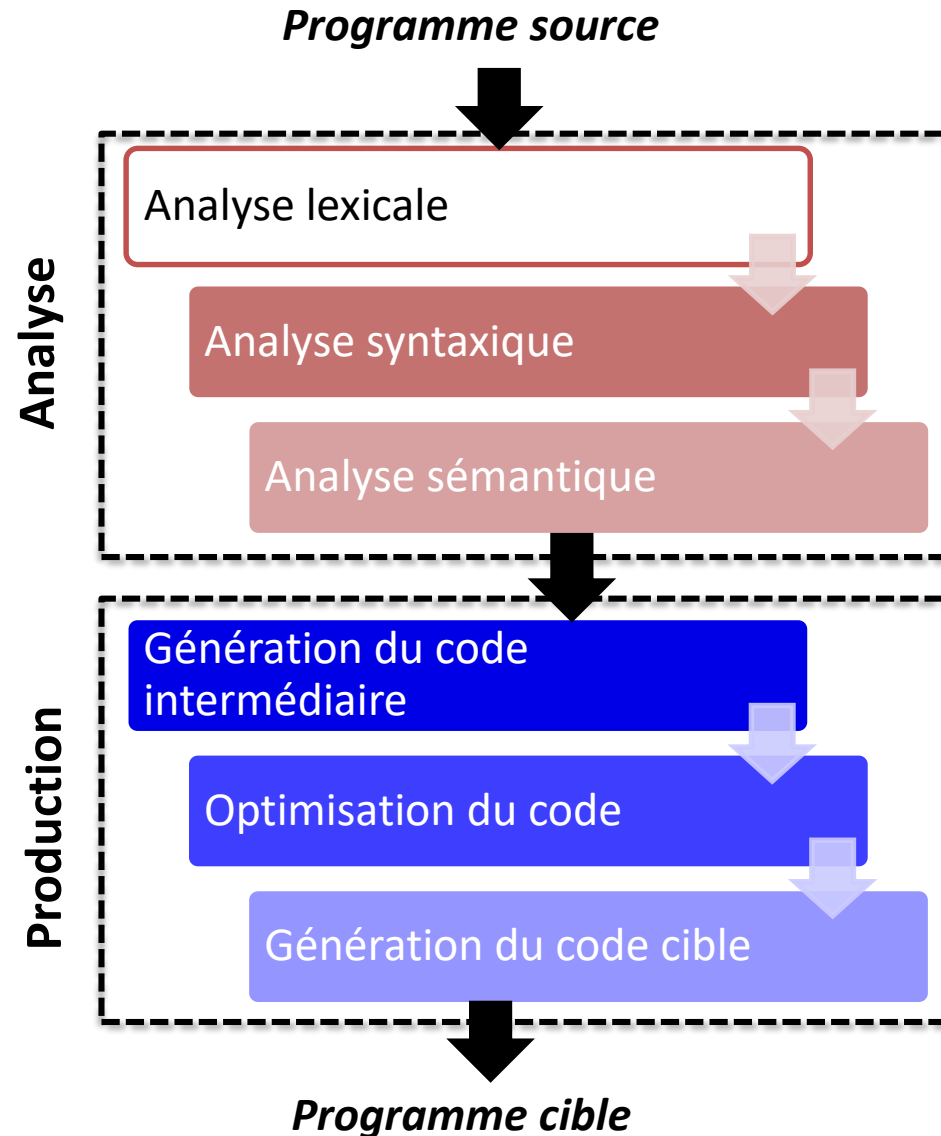
5. La structure d'un compilateur



5. La structure d'un compilateur

- La compilation se décompose en deux phases:
 - L'analyse permet de:
 - Décomposer le programme source en ses constituants, tout en associant à chacun une structure grammaticale.
 - Reconnaître les variables, les instructions, les opérateurs et élabore la structure syntaxique du programme ainsi que certaines propriétés sémantiques.
 - Créer une représentation intermédiaire du programme source
 - Accumule diverses informations dans la table de symboles
 - La production permet de:
 - construire le programme cible désiré à partir de la représentation intermédiaire et des informations de la table de symboles

5. La structure d'un compilateur



5. La structure d'un compilateur

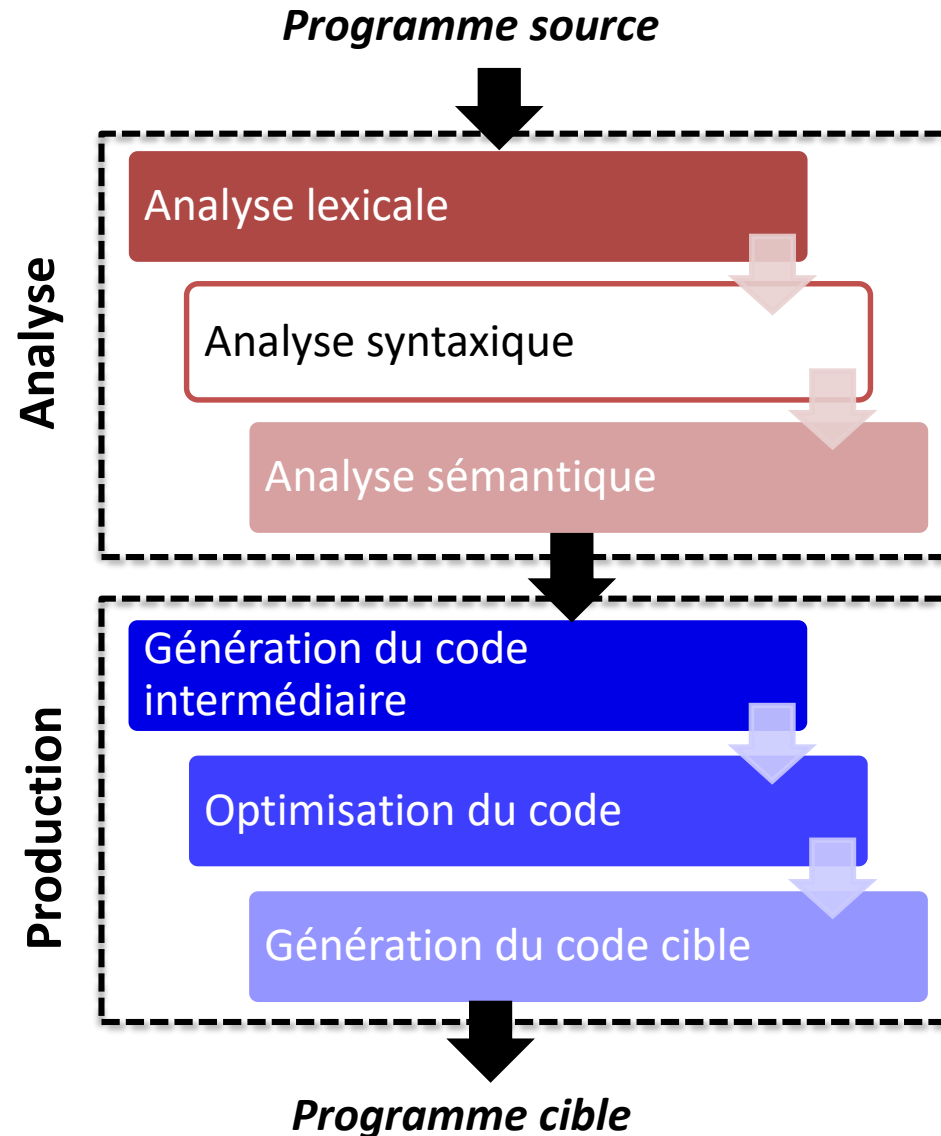
Analyse lexicale

- Les êtres humains examinent le sens de la phrase à travers sa décomposition en mots.
- Les espaces, la ponctuation et la capitalisation aident le lecteur à décomposer le groupe de lettres en une séquence de mots afin de reconnaître les mots.
- Exemple:
 - «**La jaune fille repasse l'escalier**» → Phrase composée de 6 mots: (La, jaune, fille, repasse, la, escalier).



- La première étape pour comprendre un langage c'est la **compréhension** des mots lus.

5. La structure d'un compilateur

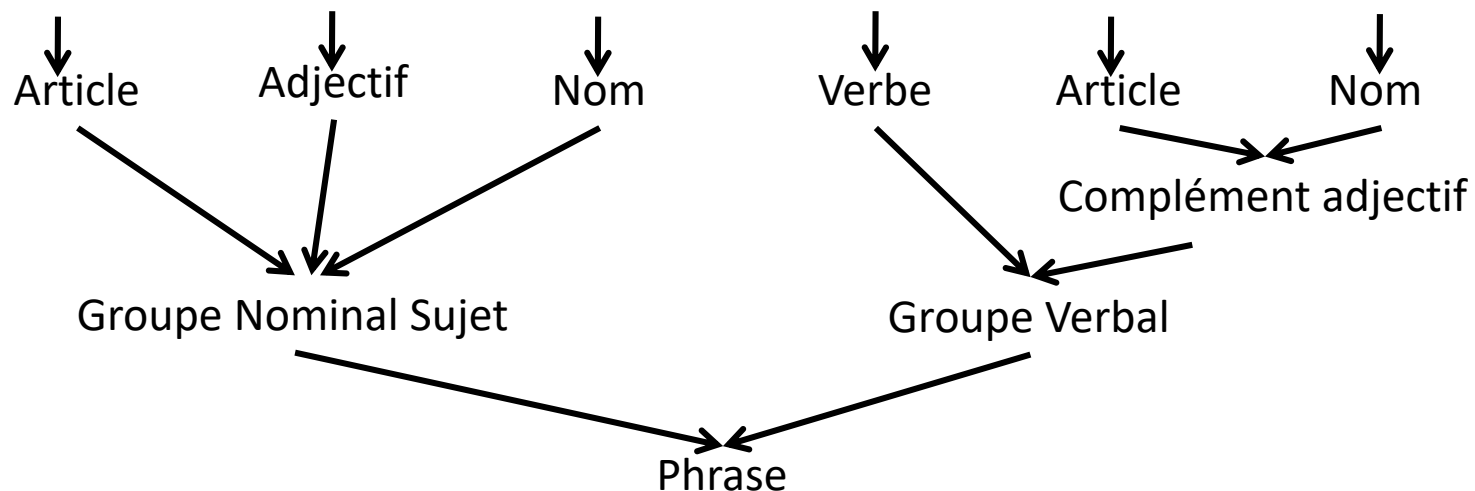


5. La structure d'un compilateur

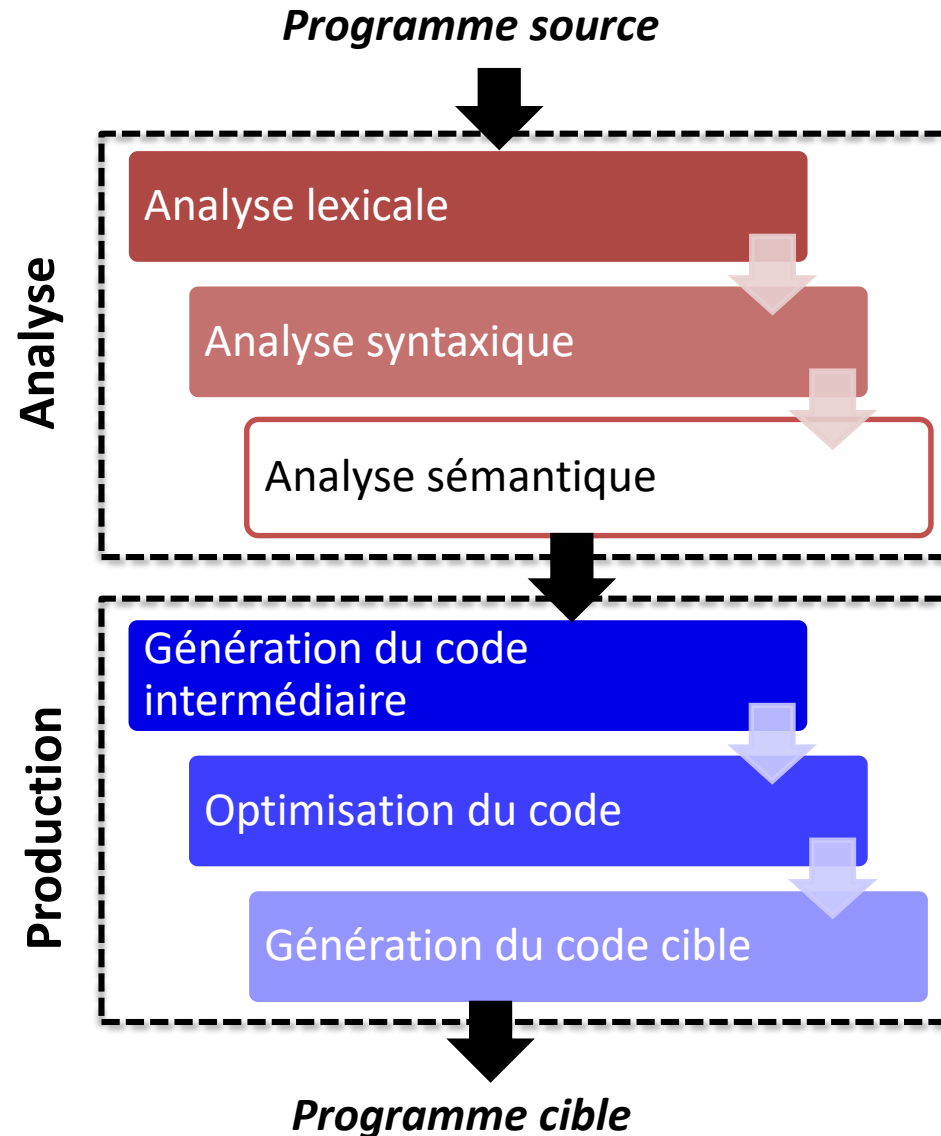
Analyse syntaxique

- Une fois les mots sont reconnus, l'étape suivante consiste à connaître la structure de la phrase :
 - Identifier le rôle de chacun des mots de la phrase
 - Grouper les mots dans des structures de plus haut niveau

« **La** **jaune** **fil****e** **re****pas****s****e** **'** **es****c****a****l****i****e****r** »



5. La structure d'un compilateur



5. La structure d'un compilateur

Analyse sémantique

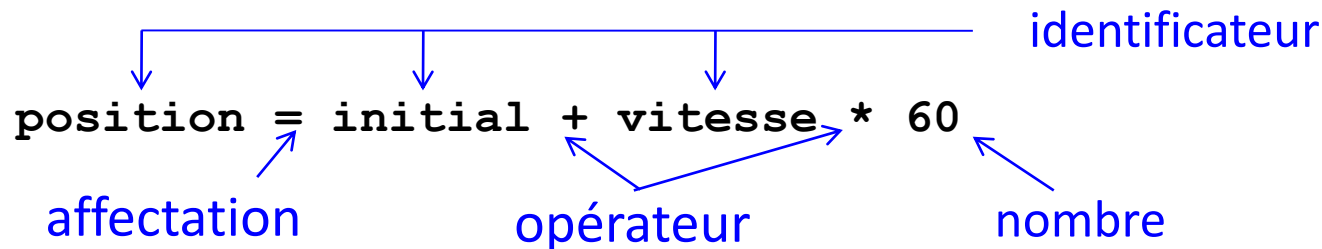
- Dans cette étape, on opère certains contrôles (contrôles de type par exemple) afin de vérifier que l'assemblage des constituants du programme a un sens et ne présente pas d'erreurs d'ordre sémantique:
 - Additionner un réel avec une chaîne de caractères.
 - Affecter une variable à un nombre, ...
- Exemple:
 - «**La jaune fille repasse l'escalier**»



5. La structure d'un compilateur

Exemple:

- L'analyse lexicale lit le flot de caractères qui constituent le programme source et les regroupe en séquence de caractères significatives appelées **lexèmes**.
- Pour chaque lexème, l'analyse lexicale émet une **unité lexicale** de la forme
[nom d'unité lexicale, valeur d'attribut]
 - Le composant **nom d'unité lexicale** est un symbole abstrait qui sera utilisé lors de l'analyse syntaxique
 - Le composant **valeur attribut** référence une entrée de la table de symboles
- Exemple:

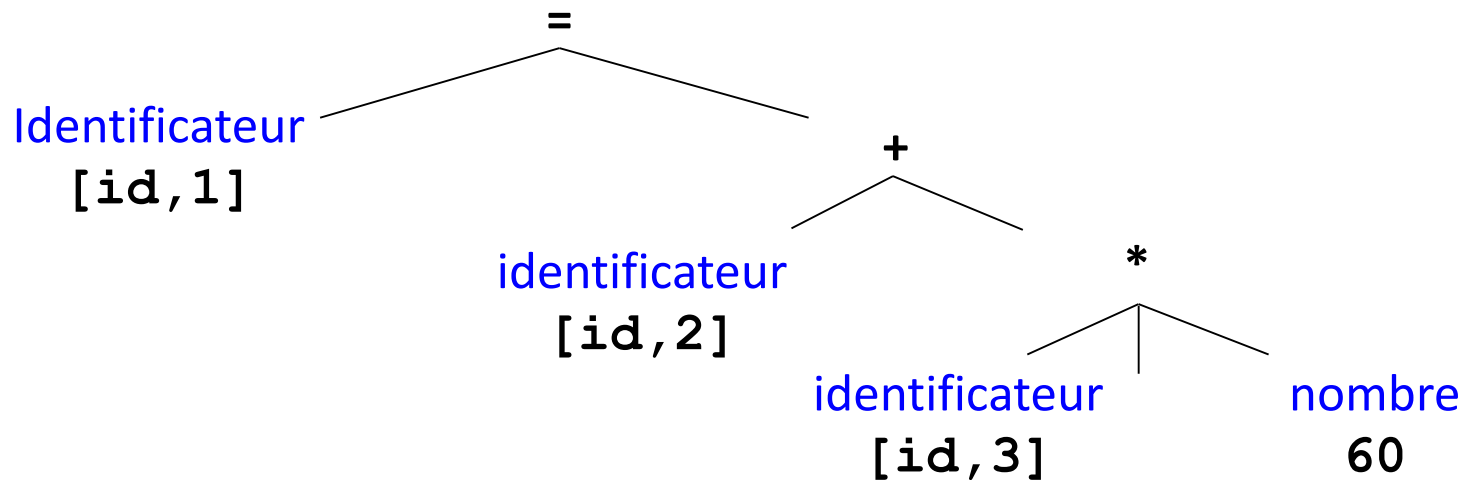


→ [id, 1] [aff, 4] [id, 2] [opp, 5] [id, 3] [opp, 6] [60]

5. La structure d'un compilateur

Exemple:

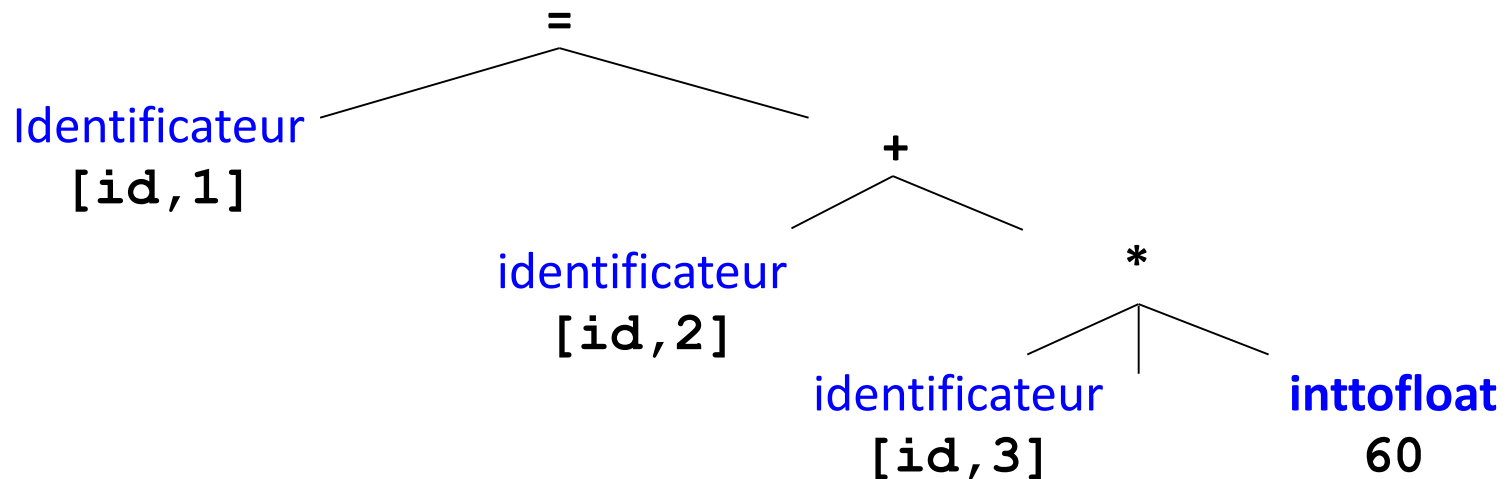
- L'analyse syntaxique se sert du premier composant des unités lexicales produites par l'analyse lexicale pour créer une représentation intermédiaire arborescente qui détient la structure grammaticale du flot de caractères.
- Exemple : L'analyse syntaxique utilise un **arbre abstrait**, dans lequel chaque nœud interne représente une opération et ses fils représentent les arguments de cette opération



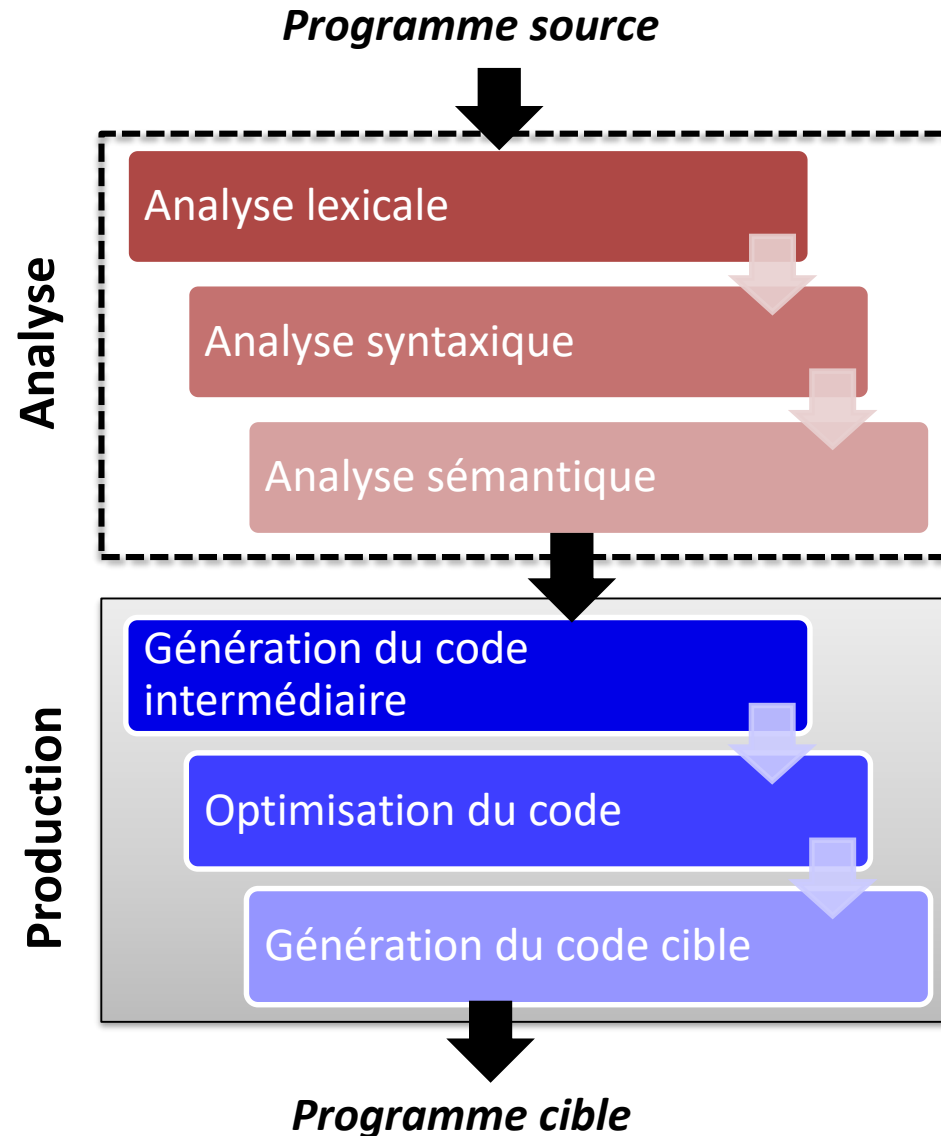
5. La structure d'un compilateur

Exemple:

- L'analyse sémantique utilise l'arbre abstrait et les informations de la table de symboles pour vérifier que le programme source est sémantiquement correct vis-à-vis de la définition du langage.
 - Elle collecte des informations sur les types et les conserve dans l'arbre abstrait ou dans la table de symboles pour la phase de production du code intermédiaire
- Exemple :



5. La structure d'un compilateur



5. La structure d'un compilateur

Exemple:

Génération du code intermédiaire

- Produire une représentation intermédiaire qui font abstraction de l'architecture de la machine cible.
- Exemple: Elle peut utilisé le ***code à trois adresses*** qui s'agit d'une suite d'instructions en un pseudo-langage d'assemblage à trois opérandes par instruction. Chaque opérande peut se comporter comme un registre

```
temp1 := inttofloat(60)
temp2 := id3 * temp1
temp3 := id2 + temp2
id1 := temp3
```

5. La structure d'un compilateur

Exemple

Optimisation du code

- Amélioration du code intermédiaire produit afin qu'il en résulte un meilleur code cible (rapide, plus court, consomme peu d'énergie, ...)
- Exemple:

```
temp1 := inttofloat(60)
temp2 := id3 * temp1
temp3 := id2 + temp2
id1 := temp3
```



```
temp1 := id3 * 60.0
id1 := id2 + temp1
```

- La constante est traduite en réel flottant à la compilation, et non à l'exécution
- La variable temp3 est éliminée

5. La structure d'un compilateur

Exemple:

Génération du code cible

- Traduire les instructions en des instructions directement exécutables par la machine réelle (sur laquelle on veut que le programme s'exécute)
- Le langage cible est dans ce cas défini par le type de processeur utilisé
- Exemple: Production du code en langage d'assemblage

```
temp1 := id3 * 60.0  
id1 := id2 + temp1
```



```
MOVF id3, R2  
MULF #60.0, R2  
MOVF id2, R1  
ADDF R2, R1  
MOVF R1, id1
```

- La première instruction transfère le contenu de id3 dans le registre R2
- La seconde multiplie le contenu du registre R2 par la constante 60.0

•Récapitulons:

Table des symboles

1	position	...
2	initial	...
3	vitesse	...
4		
5		
...		

position = initial + vitesse*60

analyseur lexical

[id,1] := [id,2] + [id,3]*60

analyseur syntaxique

=
[id,1] +
[id,2] *
[id,3] 60

analyseur sémantique

[id,1] +
[id2,] *
[id,3] inttofloat
/ 60

Phase d'analyse

MOVF id3, R2
MULF #60.0, R2
MOVF id2, R1
ADDF R2, R1
MOVF R1, id1

générateur de code cible

temp1 := id3 * 60.0
id1 := id2 + temp1

optimiseur de code

temp1 := inttofloat(60)
temp2 := id3 * temp1
temp3 := id2 + temp2
id1 := temp3

générateur de code intermédiaire

Phase de production